

Git – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 7 Sep 2026 – 13 Sep 2026 (Days 22–28)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Comprehensive reference for version control, collaboration workflows, and interview preparation.

Table of Contents

1. Core Concepts
2. Repository Setup
3. Daily Workflow
4. Staging & Commits
5. Branching Strategy
6. Merging & Rebasing
7. Remote Operations
8. Undoing Changes
9. Stash & Cherry-Pick
10. Tags & Releases
11. Git Hooks
12. Monorepo & Submodules
13. GitHub Workflow
14. Troubleshooting
15. Interview Q&A
16. Practice Resources

1. Core Concepts

Term	Meaning
Repository (repo)	Project folder tracked by Git with full history
Working Directory	Files you edit on disk
Staging Area (Index)	Snapshot of changes prepared for next commit
Commit	Immutable snapshot of staged changes with metadata
Branch	Movable pointer to a commit; enables parallel work
HEAD	Pointer to current branch/commit
Remote	Copy of repo hosted elsewhere (GitHub, GitLab)
Origin	Default name for the primary remote
Merge	Combine histories of two branches
Rebase	Replay commits on top of another branch
Conflict	Overlapping edits Git cannot auto-resolve
Detached HEAD	HEAD points to commit, not a branch

The Three Trees

Working Directory (edit) → git add → Staging Area (prepare) → git commit → Repository (history)

2. Repository Setup

```
# Initialize new repo
git init                    # creates .git/ folder
git init -b main           # start with main branch

# Clone existing repo
git clone https://github.com/user/repo.git
git clone git@github.com:user/repo.git my-folder # SSH, custom folder
git clone --depth 1 url      # shallow clone (CI, large repos)

# Configure identity (per-repo or global)
git config user.name "Your Name"
git config user.email "you@example.com"
git config --global core.editor "code --wait"
git config --global init.defaultBranch main

# View configuration
git config --list
git config user.name
```

.gitignore Essentials

```
# Dependencies
node_modules/
.pnp/

# Environment
.env
.env.local
.env.*.local

# Build output
dist/
build/
.next/
out/

# OS / IDE
.DS_Store
.idea/
.vscode/*
!.vscode/extensions.json

# Logs
*.log
npm-debug.log*

# Coverage
coverage/
.nyc_output/
```

3. Daily Workflow

```
# Check state
git status          # modified, staged, untracked
git diff            # unstaged changes
git diff --staged   # staged changes
git log --oneline --graph --all # visual history

# Stage and commit
git add src/auth.ts # specific file
git add src/         # directory
git add -p src/auth.ts # interactive hunks
git add .            # all changes (respects .gitignore)
git commit -m "feat: add JWT refresh flow"

# Push and pull
git pull origin main # fetch + merge
git pull --rebase origin main # fetch + rebase (cleaner history)
git push origin feature-auth
git push -u origin feature-auth # set upstream tracking
```

4. Staging & Commits

Commit Message Convention (Conventional Commits)

<type>(<scope>): <subject>

<body>

<footer>

Types: `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `chore`, `perf`, `ci`, `build`

Examples:

```
feat(auth): add refresh token rotation
fix(api): resolve race condition in order creation
docs(readme): add deployment instructions
refactor(user): extract validation into middleware
test(auth): add integration tests for login flow
chore(deps): upgrade express to 4.19
```

Amending Commits

```
# Add forgotten files to last commit (NOT pushed yet)
git add forgotten.ts
git commit --amend --no-edit

# Change last commit message
git commit --amend -m "feat: corrected message"

# ⚠️ Never amend commits already pushed to shared branches
```

Interactive Commit Creation

```
git commit # opens editor for full message
git commit -m "fix: typo" -m "Details here"
```

5. Branching Strategy

Git Flow

```
main (production)
├─ develop (integration)
│   ├── feature/user-auth
│   ├── feature/payment
│   └─ release/1.2.0
│       └─ hotfix/critical-bug (from main)
```

GitHub Flow (Simpler)

```
main ← always deployable
├─ feature/add-search
├─ fix/login-redirect
└─ chore/upgrade-deps
```

Trunk-Based Development

- Short-lived branches (< 1 day)
- Feature flags for incomplete work
- Frequent merges to main
- Used by Google, Meta at scale

Branch Commands

```
git branch                # list local branches
git branch -a             # include remotes
git branch feature-x      # create branch
git checkout feature-x    # switch
git switch -c feature-x   # create + switch (modern)
git branch -d feature-x   # delete merged branch
git branch -D feature-x   # force delete
git rename-branch old new # git branch -m old new
```

6. Merging & Rebasing

Merge

```
git checkout main
git merge feature-auth # creates merge commit (non-fast-forward)
git merge --no-ff feature-auth # explicit merge commit
git merge --squash feature-auth # squash all commits into one
git commit -m "feat: add auth"
```

When to merge: Shared branches, preserve full history, team prefers explicit merge commits.

Rebase

```
git checkout feature-auth
git rebase main           # replay feature commits on top of main
git rebase -i HEAD~3      # interactive: squash, reword, drop
```

When to rebase: Clean linear history, local/unpushed work, before opening PR.

Golden Rule: Never rebase commits that others have based work on.

Merge Conflicts

```
git merge feature-x
# CONFLICT in src/app.ts

# 1. Open file, resolve markers:
# <<<<<<< HEAD
# your changes
# =====
# their changes
# >>>>>>> feature-x

# 2. Stage resolved files
git add src/app.ts
git merge --continue      # or git commit for merge

# Abort if needed
git merge --abort
git rebase --abort
```

7. Remote Operations

```
git remote -v                # list remotes
git remote add upstream url   # add fork upstream
git fetch origin              # download commits, don't merge
git fetch --all --prune       # fetch all, remove stale refs

# Track remote branch
git checkout -b feature-x origin/feature-x
git branch --set-upstream-to=origin/main main

# Force push (DANGEROUS - team coordination required)
git push --force-with-lease origin feature-x  # safer than --force
```

Fork Workflow

```
# 1. Fork on GitHub
# 2. Clone your fork
git clone git@github.com:you/repo.git
cd repo
git remote add upstream git@github.com:original/repo.git

# 3. Sync with upstream
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

8. Undoing Changes

Goal	Command
Discard unstaged edits	<code>git restore file.js</code>
Unstage file	<code>git restore --staged file.js</code>
Undo last commit, keep changes	<code>git reset --soft HEAD~1</code>
Undo last commit, discard changes	<code>git reset --hard HEAD~1</code>
Revert a commit (safe for shared)	<code>git revert abc123</code>
Find lost commits	<code>git reflog</code>
Restore deleted file	<code>git checkout HEAD -- file.js</code>

```
# Reflog – your safety net
git reflog
git reset --hard HEAD@{2}          # go back to reflog entry

# Revert creates new commit undoing changes
git revert abc123 --no-edit
git revert HEAD~3..HEAD           # revert range
```

9. Stash & Cherry-Pick

Stash

```
git stash                        # save WIP, clean working tree
git stash push -m "WIP auth refactor"
git stash list
git stash pop                   # apply and remove
git stash apply stash@{1}       # apply, keep stash
git stash drop stash@{0}
git stash clear
```

Cherry-Pick

```
# Apply specific commit(s) to current branch
git cherry-pick abc123
git cherry-pick abc123 def456
git cherry-pick -n abc123          # stage without committing
```

10. Tags & Releases

```
# Lightweight tag
git tag v1.0.0

# Annotated tag (recommended for releases)
git tag -a v1.0.0 -m "Release 1.0.0"
git tag -a v1.0.0 abc123          # tag specific commit

git push origin v1.0.0
git push origin --tags

# Delete tag
git tag -d v1.0.0
git push origin --delete v1.0.0
```

Semantic Versioning: **MAJOR.MINOR.PATCH** — breaking, feature, fix.

11. Git Hooks

Located in **.git/hooks/** or managed via Husky.

```
# pre-commit: lint, format, type-check
#!/bin/sh
npm run lint-staged

# commit-msg: validate conventional commits
#!/bin/sh
npx commitlint --edit $1

# pre-push: run tests
#!/bin/sh
npm test
```

Husky setup:

```
npm install -D husky lint-staged @commitlint/cli @commitlint/config-conventional
npx husky init
echo "npx lint-staged" > .husky/pre-commit
```

12. Monorepo & Submodules

Submodules

```
git submodule add https://github.com/org/lib.git libs/lib
git submodule update --init --recursive
git clone --recurse-submodules url
```

Monorepo (common in JS)

- Turborepo, Nx, Lerna for multi-package repos
- Single repo, shared CI, atomic cross-package changes

13. GitHub Workflow

Pull Request Checklist

1. Create feature branch from `main`
2. Make focused commits with clear messages
3. Push branch, open PR with description
4. Link issue: `Closes #42`
5. Request review, address feedback
6. Ensure CI passes
7. Squash merge or merge commit per team policy
8. Delete branch after merge

PR Template

Summary

- What changed and why

Test plan

- [] Unit tests pass
- [] Manual testing steps

Screenshots (if UI)

Code Review Etiquette

- Review code, not the person
- Ask questions, suggest alternatives
- Approve when ready; request changes when blocking
- Keep PRs small (< 400 lines ideal)

14. Troubleshooting

```
# "Your branch and origin/main have diverged"
git pull --rebase origin main

# Accidentally committed to main
git branch backup-main
git reset --hard origin/main
git checkout -b feature/fix
git cherry-pick backup-main

# Large file committed (use Git LFS)
git lfs install
git lfs track "*.psd"
git add .gitattributes

# Clean untracked files
git clean -fd                                # dry run: git clean -fdn

# Bisect - find breaking commit
git bisect start
git bisect bad                                # current is broken
git bisect good v1.0.0                        # this version worked
# test, then git bisect good/bad until found
git bisect reset
```

15. Interview Q&A

Q: Difference between merge and rebase? A: Merge combines histories with a merge commit; rebase replays commits linearly. Merge preserves context; rebase creates cleaner history but rewrites commits.

Q: What is a detached HEAD? A: HEAD points directly to a commit instead of a branch. New commits aren't on any branch until you create one with `git switch -c rescue`.

Q: How do you undo a pushed commit? A: Use `git revert` to create a new commit that undoes changes. Avoid `reset --hard` + force push on shared branches.

Q: What is `git fetch` vs `git pull`? A: Fetch downloads remote changes without merging. Pull = fetch + merge (or rebase with `--rebase`).

Q: Explain fast-forward merge. A: When target branch hasn't diverged, Git moves the pointer forward without a merge commit.

Q: What is `.gitignore` vs `.git/info/exclude`? A: `.gitignore` is shared in repo; `exclude` is local-only, not committed.

Q: How do you resolve merge conflicts in a large team? A: Communicate ownership, pull frequently, use small PRs, run merge tools (`git mergetool`), agree on conflict resolution in code review.

Q: What is git stash used for? A: Temporarily save uncommitted work to switch branches without committing incomplete changes.

Q: Difference between `git reset --soft` , `--mixed` , `--hard`? A: Soft keeps staged + working; mixed (default) keeps working only; hard discards everything.

Q: What is cherry-pick? A: Apply a specific commit from one branch onto another without merging entire branch.

16. Practice Resources

Resource	URL	Focus
Learn Git Branching	learngitbranching.js.org	Interactive visual
Pro Git Book	git-scm.com/book	Official reference
GitHub Skills	skills.github.com	Hands-on labs
Atlassian Git Tutorials	atlassian.com/git/tutorials	Workflows
Oh Shit Git	ohshitgit.com	Recovery scenarios

Practice Exercises

1. Create repo, make 5 commits, use `git log --oneline --graph`
2. Create two branches with conflicting edits, resolve merge conflict
3. Rebase feature branch onto main, squash commits with `-i`
4. Use stash to switch branches mid-work
5. Fork a repo, add upstream, sync, submit PR
6. Use reflog to recover "lost" commit after hard reset
7. Set up Husky pre-commit hook with lint-staged
8. Tag a release and push tags to remote

Quick Reference Card

```
git status | diff | log --oneline --graph  
git add -p | commit -m "type: msg" | push  
git branch | switch -c | merge | rebase  
git fetch | pull --rebase | push -u  
git restore | reset | revert | reflog  
git stash | cherry-pick | tag
```

Remember: Commit often, push daily, pull before push, never force push to main.

Product Company Interview Bank – Git Version Control (50 Questions)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Target companies: Google, Meta, Microsoft, Amazon, Adobe, Atlassian, Uber, Flipkart, Salesforce, Stripe **Rule:** Full terms only — no unexplained abbreviations. Write for absolute beginners who know nothing.

Question 1: What is Git, and why do product companies use it?

Answer: Git is a distributed version control system that records every change to your project files over time. Product companies use Git because many engineers can work on the same codebase in parallel without overwriting each other's work. Git keeps a complete history of changes, makes code review possible before release, and allows teams to roll back broken changes quickly. At companies like Google or Amazon, Git (or Git-based tools) sits at the center of daily development, continuous integration, and deployment pipelines.

Question 2: What is the difference between Git and GitHub?

Answer: Git is the version control software that runs on your computer and tracks file history locally. GitHub is a cloud hosting service built on top of Git that stores repositories remotely and adds collaboration features like pull requests, issue tracking, and access control. You can use Git entirely offline on your machine; GitHub requires an internet connection to sync and collaborate. Think of Git as the engine and GitHub as a garage where teams park and share their projects.

Question 3: What happens when you run `git init` ?

Answer: The command `git init` creates a new Git repository in your current folder by adding a hidden `.git` directory. That directory stores all commits, branches, tags, and configuration for the project. After initialization, Git starts tracking changes only after you explicitly add files

with `git add`. Running `git init` does not connect to any remote server; it only sets up local version control.

Question 4: Explain the three areas in Git: working directory, staging area, and repository.

Answer: The working directory is where you edit files on disk. The staging area (also called the index) is a middle layer where you place changes you intend to include in the next commit. The repository is the stored history of committed snapshots inside the `.git` folder. A typical flow is: edit files in the working directory, use `git add` to stage them, then use `git commit` to save them permanently in the repository.

Question 5: What is a commit, and what information does it contain?

Answer: A commit is a saved snapshot of your project at a specific point in time. Each commit contains a unique identifier (a 40-character hash), a pointer to its parent commit, the author name, the author email, a timestamp, and a commit message describing the change. Commits are immutable once created; you do not edit old commits—you create new ones. Good commit messages at product companies explain *why* a change was made, not just *what* changed.

Question 6: What does `git status` tell you?

Answer: The command `git status` shows the current state of your working directory and staging area relative to the last commit. It lists untracked files (new files Git has never seen), modified files not yet staged, and staged files ready to commit. It also tells you which branch you are on and whether it is ahead of or behind the remote branch. Engineers run `git status` constantly before committing to avoid surprises.

Question 7: What is the difference between `git add .` and `git add -p`?

Answer: The command `git add .` stages all new and modified files in the current directory and subdirectories at once. The command `git add -p` (patch mode) lets you review each change hunk by hunk and choose which parts to stage. Patch mode is preferred at product companies when a single file contains multiple unrelated changes that should become separate commits. Smaller, focused commits make code review and bug investigation much easier.

Question 8: How do you write a good commit message?

Answer: A good commit message has a short summary line (under 50–72 characters) in imperative mood, such as “Fix login timeout on slow networks.” Optionally add a blank line and a longer body explaining motivation, approach, and trade-offs. Avoid vague messages like “fix bug” or “update stuff.” At Stripe or Salesforce, reviewers often reject pull requests whose commit messages do not explain the business or technical reason for the change.

Question 9: What is a branch in Git?

Answer: A branch is a movable pointer to a specific commit, allowing parallel lines of development within the same repository. The default branch is usually named `main` or `master`. When you create a branch, Git copies the pointer so you can make commits without affecting other branches. Branches are cheap in Git—they are just small reference files, not full copies of your codebase.

Question 10: How do you create and switch to a new branch?

Answer: Create a branch with `git branch feature-name`, which adds the pointer but does not switch to it. Switch with `git checkout feature-name` or use the combined command `git checkout -b feature-name` to create and switch in one step. Modern Git also supports `git switch -c feature-name` for the same purpose. At product companies, branches are typically named after ticket numbers or features, such as `JIRA-1234-add-payment-retry`.

Question 11: What is the difference between `git merge` and `git rebase`?

Answer: Merge combines two branch histories by creating a new merge commit that has two parents, preserving the exact timeline of when work happened. Rebase replays your commits on top of another branch’s tip, producing a linear history without a merge commit. Merge is safer for shared branches because it does not rewrite history. Rebase keeps history clean but must never be used on commits already pushed and shared with teammates unless the team explicitly agrees.

Question 12: When would you choose merge over rebase in a product team?

Answer: Choose merge when integrating feature branches into a shared main branch, especially if multiple people worked on the feature branch or if the branch was already pushed publicly. Merge preserves the original commit timestamps and makes it clear when integration happened. Most companies require merge (or squash merge via the hosting platform) for pull requests into protected main branches. Rebase is typically reserved for cleaning up your own local commits before opening a pull request.

Question 13: What is a merge conflict, and why does it happen?

Answer: A merge conflict occurs when Git cannot automatically combine changes because two commits modified the same lines in the same file differently. Git stops the merge and marks the conflicting sections in the file with markers like `<<<<<<<`, `=====`, and `>>>>>>>`. Conflicts are normal when two engineers edit the same code region. They are not errors—they are Git asking a human to decide the correct final content.

Question 14: Walk through resolving a merge conflict step by step.

Answer: First, run `git status` to see which files conflict. Open each conflicted file and find the conflict markers. Edit the file to keep the correct code and remove all markers. Save the file, then run `git add filename` to mark it resolved. After all conflicts are fixed and staged, run `git commit` (or `git merge --continue` if a merge was in progress). Finally, run tests locally before pushing, because conflict resolution can accidentally drop logic from either side.

Question 15: What does `git cherry-pick` do, and when is it used?

Answer: Cherry-pick applies the changes from a specific commit onto your current branch as a new commit with a different identifier. It is used when you need one fix from another branch without merging the entire branch. For example, a critical bug fix landed on the main branch and must also go to a release branch. Use cherry-pick carefully—if the original commit depends on earlier commits, cherry-picking may cause new conflicts or broken builds.

Question 16: What is `git stash`, and when should you use it?

Answer: Stash temporarily saves uncommitted changes (both staged and unstaged) and reverts your working directory to a clean state matching the last commit. Use it when you must switch branches urgently but your current work is not ready to commit. Restore stashed work with `git stash pop` (apply and remove) or `git stash apply` (apply but keep the stash). At busy product teams, stash is common when a production incident interrupts feature development.

Question 17: What is the difference between `git reset`, `git revert`, and `git restore`?

Answer: `git reset` moves the current branch pointer backward and can discard commits from branch history (dangerous on shared branches). `git revert` creates a new commit that undoes a previous commit's changes, preserving history—this is the safe choice on shared main branches. `git restore` (modern Git) discards uncommitted changes in the working directory or unstages files without rewriting commit history. On protected main branches at Amazon or Microsoft, revert is almost always preferred over reset.

Question 18: Explain soft, mixed, and hard reset.

Answer: `git reset --soft HEAD~1` moves the branch pointer back one commit but keeps changes staged—useful for redoing a commit message. `git reset --mixed HEAD~1` (the default) moves the pointer back and unstages changes, but files remain modified in the working directory. `git reset --hard HEAD~1` moves the pointer back and discards all uncommitted changes permanently. Hard reset is destructive and should never be used on commits that teammates rely on.

Question 19: What is the reflog, and why is it a lifesaver?

Answer: The reflog (reference log) records every movement of branch pointers and HEAD in your local repository, including resets, rebases, and checkouts. Even if you accidentally run a hard reset and “lose” commits, `git reflog` shows the old commit identifiers so you can recover them with `git checkout` or `git reset --hard` to that identifier. Reflog entries expire after a default period (usually 90 days). It is local only—there is no reflog on the remote server for other people's machines.

Question 20: How do remotes work in Git?

Answer: A remote is a named reference to another copy of the repository, usually on a server like GitHub or GitLab. The default remote is typically named `origin`. You fetch updates with `git fetch origin`, which downloads new commits without changing your working files. You push your commits with `git push origin branch-name`, which uploads your local branch to the remote. Remotes enable collaboration: each engineer has a full local copy and syncs through the shared remote.

Question 21: What is the difference between `git fetch`, `git pull`, and `git push`?

Answer: `git fetch` downloads new commits and branch updates from the remote but does not merge them into your current branch—you inspect first, then merge manually. `git pull` is fetch plus merge (or fetch plus rebase if configured) in one command—it updates your current branch immediately. `git push` uploads your local commits to the remote branch. At product companies, some teams prefer `git fetch` followed by explicit merge or rebase so pulls never surprise you with automatic merges.

Question 22: What is a pull request, and how does the review flow work?

Answer: A pull request (also called a merge request on GitLab) is a formal proposal to merge your branch into a target branch like `main`. You push your branch, open the pull request on GitHub or GitLab, and request reviewers from your team. Reviewers leave comments, request changes, or approve. Continuous integration checks run automatically on the pull request. Once approved and checks pass, the branch is merged—often with squash merge to keep main history clean.

Question 23: What is a protected branch, and why do companies use them?

Answer: A protected branch is a branch (usually `main` or `production`) with rules enforced by the hosting platform. Rules commonly include: no direct pushes, required pull request approvals, required passing tests, and required signed commits. Protected branches prevent accidental force pushes and untested code from reaching production. At Flipkart or Uber, pushing directly to main is blocked—every change must go through the pull request pipeline.

Question 24: What is force push, and when is it dangerous?

Answer: Force push (`git push --force`) overwrites the remote branch history with your local history, discarding commits that exist on the remote but not locally. It is dangerous on shared branches because teammates' work can vanish from the remote history. Force push is occasionally acceptable on your own feature branch after a local rebase, often using the safer `git push --force-with-lease` which fails if someone else pushed in the meantime. Never force push to main or any branch others are actively using.

Question 25: What does `git clone` do compared to `git fork`?

Answer: `git clone` creates a full local copy of a repository on your machine, including all branches and history. A fork (on GitHub or GitLab) creates your own copy of someone else's repository under your account on the server—you then clone your fork locally. Forking is standard for open-source contributions where you do not have write access to the original repository. You push to your fork and open a pull request back to the original project.

Question 26: What is `.gitignore`, and what should you put in it?

Answer: The `.gitignore` file lists file and folder patterns that Git should never track. Common entries include dependency folders (`node_modules/`), build output (`dist/`, `build/`), environment files (`.env`), operating system files (`.DS_Store`), and log files. Ignoring secrets and generated artifacts keeps repositories clean and prevents accidental credential leaks. At product companies, committing secrets to Git is a serious security incident—`.gitignore` is the first line of defense, not the only one.

Question 27: How do you remove a file from Git history that was committed by mistake?

Answer: If the file was just committed locally and not pushed, you can use `git reset` or amend the commit to remove it, then add the file to `.gitignore`. If it was already pushed—especially if it contains secrets—you must treat it as a security incident: rotate the exposed credentials immediately and use tools like `git filter-repo` or BFG Repo-Cleaner to purge the file from all history. Simply deleting the file in a new commit does not remove it from old commits that attackers can still read.

Question 28: What are Git tags, and how do they differ from branches?

Answer: Tags are permanent pointers to specific commits, typically used to mark release versions like `v1.0.0` or `v2.3.1`. Unlike branches, tags do not move forward as you make new commits—they stay fixed on one commit forever. Lightweight tags are simple names; annotated tags store extra metadata like tagger name, date, and message. Product teams tag releases so operations and support can always check out the exact code that is running in production.

Question 29: How do you create and push an annotated tag for a release?

Answer: Create an annotated tag with `git tag -a v1.0.0 -m "Release version 1.0.0"`. Push it to the remote with `git push origin v1.0.0` or push all tags with `git push origin --tags`. Annotated tags are preferred for releases because they include metadata and are cryptographically signable. Release pipelines at companies like Adobe often trigger automatically when a version tag matching a pattern (such as `v*.*.*`) is pushed.

Question 30: What are Git hooks, and give examples used in product teams?

Answer: Git hooks are scripts that run automatically at specific Git events, such as before a commit or before a push. A `pre-commit` hook might run a linter or formatter to block commits with style violations. A `commit-msg` hook might enforce a message format like `JIRA-1234: description`. A `pre-push` hook might run unit tests locally before code reaches the remote. Teams often manage hooks through tools like Husky (for JavaScript projects) or pre-commit (for Python) so hooks are shared via the repository.

Question 31: What is `git bisect`, and how does it help debug regressions?

Answer: Git bisect performs a binary search through commit history to find the exact commit that introduced a bug. You mark a known good commit as `git bisect good` and a known bad commit as `git bisect bad`, then Git checks out the middle commit for you to test. You repeat marking good or bad until Git identifies the first bad commit. At Meta-scale codebases, bisect saves hours compared to manually checking every commit between two release points.

Question 32: What is a submodule, and when would a team use one?

Answer: A submodule embeds one Git repository inside another as a subdirectory, pinned to a specific commit of the inner repository. Teams use submodules when a shared library lives in its own repository but must be included at an exact version inside a larger project. The parent repository stores a pointer to the submodule commit, not the submodule's full history.

Submodules add complexity—every clone requires `git submodule update --init --recursive`—so many teams prefer package managers or monorepos instead.

Question 33: What is a monorepo, and how does Git support it?

Answer: A monorepo is a single Git repository containing multiple projects, services, or packages instead of many separate repositories. Git supports monorepos natively because branches, tags, and history work the same regardless of repository size. Challenges include slow clones, noisy commit history across unrelated teams, and expensive continuous integration. Companies like Google and Meta use monorepos with custom tooling (partial clones, sparse checkout, affected-target testing) to keep workflows manageable at scale.

Question 34: What is sparse checkout in Git?

Answer: Sparse checkout lets you clone or check out only specific directories from a large repository instead of the entire tree. This reduces disk usage and speeds up operations in monorepos where you only work on one service. Enable it with `git sparse-checkout init --cone` and specify folders with `git sparse-checkout set path/to/folder`. Sparse checkout is increasingly important as monorepos grow—engineers at Microsoft and Google rely on it daily.

Question 35: What is the difference between squash merge and regular merge in a pull request?

Answer: Regular merge preserves every commit from the feature branch and adds a merge commit connecting the histories. Squash merge combines all feature branch commits into a single new commit on the target branch, discarding individual commit messages except the squash summary. Squash merge keeps main branch history linear and readable—one pull request equals one commit. Many product companies enforce squash merge on main so bisect and revert operations map cleanly to one pull request.

Question 36: How do you undo the last commit that was already pushed to a shared branch?

Answer: Use `git revert HEAD` to create a new commit that reverses the last commit's changes, then push normally—no force push needed. If you must revert an older commit, use `git revert <commit-hash>`. Revert is safe because it adds history instead of deleting it, so teammates' clones stay compatible. Reset and force push on shared branches is almost never acceptable at product companies and may violate compliance policies.

Question 37: You ran `git pull` and got a merge conflict. What do you do?

Answer: Do not panic—run `git status` to list conflicted files. Open each file, resolve conflict markers manually, and save. Stage resolved files with `git add`. Complete the merge with `git commit` (Git pre-fills a merge commit message). Run your test suite to verify nothing broke during resolution. If the conflict is too messy, you can abort with `git merge --abort` to return to the pre-pull state and ask the teammate who owns the conflicting code for help.

Question 38: What is `git blame`, and how should you use it professionally?

Answer: `git blame` (or `git annotate`) shows who last modified each line of a file and in which commit. It helps trace when a bug was introduced and who to ask for context. Use it for investigation, not for assigning public blame—many teams prefer the neutral term `git log -L` or hosting-platform “blame” views. At code review, blame helps you understand whether a suspicious line is legacy code or a recent regression.

Question 39: What is a detached HEAD state, and how do you fix it?

Answer: Detached HEAD means Git's HEAD pointer sits directly on a commit rather than on a branch name—common after checking out a specific tag or commit hash. Commits made in detached HEAD are not attached to any branch and can become unreachable once you switch away. Fix it by creating a branch: `git switch -c rescue-branch` before making commits, or switch back to an existing branch with `git switch main`. Always check `git status`—it warns you when HEAD is detached.

Question 40: Explain the typical Git workflow for a feature from start to merge.

Answer: Sync main with `git pull origin main`, then create a feature branch with `git switch -c feature/add-search`. Make small commits with clear messages, pushing regularly with `git push -u origin feature/add-search`. Open a pull request, address review comments with additional commits, and wait for continuous integration to pass. After approval, squash merge into main on the hosting platform, delete the feature branch, and pull the updated main locally. This trunk-based or GitHub-flow pattern is standard at Stripe, Salesforce, and most product companies.

Question 41: What is signing commits, and why do some companies require it?

Answer: Commit signing uses a cryptographic key (GPG or SSH) to prove that a commit was actually created by the claimed author. GitHub and GitLab display a “Verified” badge on signed commits. Companies require signing to prevent impersonation and to meet supply-chain security standards. Configure it once with `git config commit.gpgsign true` and upload your public key to your hosting account. Unsigned commits on protected branches may be rejected automatically.

Question 42: What is the difference between GitLab merge requests and GitHub pull requests?

Answer: Functionally they are the same concept—proposing to merge one branch into another with code review and automation. GitHub calls them pull requests; GitLab calls them merge requests. GitLab often integrates merge request approvals, environments, and deployment tracking in one interface. GitHub relies heavily on Actions for continuous integration and third-party apps. Interviewers may ask this to check whether you understand that the workflow is universal even if the product names differ.

Question 43: How do you compare changes between branches or commits?

Answer: Use `git diff main..feature-branch` to see all code differences between two branches. Use `git diff HEAD~1` to compare the last commit with its parent. Use `git log main..feature-branch` to list commits on the feature branch that are not yet on main. On

GitHub, the pull request “Files changed” tab shows the same diff visually. Before merging, reviewers rely on diffs to catch unintended changes, deleted tests, or missing migrations.

Question 44: What is `git clean`, and when would you use it?

Answer: `git clean` removes untracked files and directories from the working directory—files Git is not managing at all. Use `git clean -fd` to delete untracked files and folders after confirming you do not need them (dry run first with `git clean -n`). This is useful when build artifacts clutter the tree and `.gitignore` missed them, or when switching experiments. It does not remove tracked files; use `git restore` or `git reset` for those.

Question 45: How do you handle a situation where your feature branch is many commits behind main?

Answer: First, fetch latest main with `git fetch origin main`. Then either merge main into your branch with `git merge origin/main` (preserves history, adds a merge commit) or rebase onto main with `git rebase origin/main` (linear history, rewrites your commits). After rebase, you must force push your feature branch with `--force-with-lease`. Resolve any conflicts during merge or rebase, run tests, and push. Keeping branches updated reduces painful mega-conflicts on pull request merge day.

Question 46: What are common mistakes junior engineers make with Git in interviews?

Answer: Committing secrets or large binary files, force pushing to shared branches, making giant commits that mix unrelated changes, and using hard reset on pushed work. Another mistake is rebasing commits that teammates already pulled, which duplicates work and confuses history. Interviewers also watch for inability to explain merge versus rebase trade-offs or not knowing how to recover from a bad reset using reflog. Demonstrating calm conflict resolution and revert-over-reset judgment signals mid-level readiness.

Question 47: How does Git integrate with continuous integration pipelines?

Answer: Continuous integration runs automated builds and tests on every push or pull request. Hosting platforms send webhooks to services like GitHub Actions, GitLab Continuous Integration, Jenkins, or CircleCI when code changes. The pipeline clones the repository at the

pull request commit, installs dependencies, runs linters and tests, and reports pass or fail status back to the pull request. Merging is blocked until required checks pass. At Amazon and Microsoft, this gatekeeping is non-negotiable for production branches.

Question 48: What is `git worktree`, and when is it useful?

Answer: `git worktree` lets you check out multiple branches simultaneously in separate directories linked to the same repository. Instead of stashing and switching, you can have `main` in one folder and `hotfix/bug` in another and work on both in parallel. This is useful when a production hotfix interrupts feature work—you fix the hotfix in the second worktree without disturbing uncommitted feature changes. Each worktree shares the same `.git` database but has its own working directory.

Question 49: How would you explain Git to a teammate who keeps breaking the build with bad merges?

Answer: I would pair with them on a safer workflow: always pull latest main before starting work, make small commits, push the feature branch early, and open a draft pull request so continuous integration catches issues quickly. Teach them to run `git status` and tests before every push, use merge instead of rebase on shared branches until comfortable, and never force push to main. If they force pushed or reset badly, show them reflog recovery once—it builds confidence that Git is recoverable when used carefully.

Question 50: A production bug exists in the latest release. How does Git help you hotfix it?

Answer: Check out the release tag (for example `v2.1.0`) or the release branch, create a hotfix branch with `git switch -c hotfix/payment-crash v2.1.0`, fix the bug, and commit. Open an urgent pull request, get expedited review, merge, and tag the new release (`v2.1.1`). Cherry-pick the fix onto main if it is not already there so the bug does not return in the next release. Git's tags and branches let operations deploy an exact hotfix while feature development continues unaffected on main.